

Week 12 Practical

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and System throughput under different buffer sizes.

```
divya@Divyasree:~$ mkdir pc_semaphore
divya@Divyasree:~$ cd pc_semaphore
divya@Divyasree:~/pc_semaphore$ pwd
/home/divya/pc_semaphore
divya@Divyasree:~/pc_semaphore$ nano pc_semaphore.c
divya@Divyasree:~/pc_semaphore$ ls
pc_semaphore.c
divya@Divyasree:~/pc_semaphore$ gcc pc_semaphore.c -o pc_semaphore -pthread
divya@Divyasree:~/pc_semaphore$ ls
pc_semaphore  pc_semaphore.c
divya@Divyasree:~/pc_semaphore$ ./pc_semaphore 5
```

```
=====
PRODUCER-CONSUMER RESULTS
=====
Buffer Size      : 5
Items Produced   : 100000
Items Consumed   : 100000
Execution Time    : 0.737644 seconds
Throughput       : 135566.81 items/second
Synchronization : CORRECT
divya@Divyasree:~/pc_semaphore$ ./pc_semaphore 10
```

```
=====
PRODUCER-CONSUMER RESULTS
=====
Buffer Size      : 10
Items Produced   : 100000
Items Consumed   : 100000
Execution Time    : 0.360260 seconds
Throughput       : 277577.05 items/second
Synchronization : CORRECT
```

```
divya@Divyasree:~/pc_semaphore$ ./pc_semaphore 20
```

```
=====
PRODUCER-CONSUMER RESULTS
=====
Buffer Size      : 20
Items Produced   : 100000
Items Consumed   : 100000
Execution Time    : 0.192925 seconds
Throughput       : 518335.62 items/second
Synchronization : CORRECT
```

```
divya@Divyasree:~/pc_semaphore$ ./pc_semaphore 50
```

```
=====
PRODUCER-CONSUMER RESULTS
=====
Buffer Size      : 50
Items Produced   : 100000
Items Consumed   : 100000
Execution Time    : 0.133537 seconds
Throughput       : 748853.55 items/second
Synchronization : CORRECT
divya@Divyasree:~/pc_semaphore$ |
```

Code:

```
GNU nano 7.2 pc_semaphore.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>

#define ITEMS 100000

int *buffer;
int buffer_size;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

long long produced = 0;
long long consumed = 0;

/* Producer Thread */
void *producer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        /* Wait for an empty slot */
        sem_wait(&empty);


```

```
GNU nano 7.2 pc_semaphore.c
    }

    return NULL;
}

/* Consumer Thread */
void *consumer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        /* Wait for an available item */
        sem_wait(&full);

        /* Enter critical section */
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        (void)item;

        out = (out + 1) % buffer_size;
        consumed++;

        pthread_mutex_unlock(&mutex);

        /* Signal that an empty slot is available */
        sem_post(&empty);
    }

    return NULL;
}
```

```

GNU nano 7.2                                     pc_semaphore.c
}

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: %s <buffer_size>\n", argv[0]);
        return 1;
    }

    buffer_size = atoi(argv[1]);

    if (buffer_size <= 0)
    {
        printf("Buffer size must be greater than 0.\n");
        return 1;
    }

    /* Allocate buffer */
    buffer = malloc(buffer_size * sizeof(int));

    if (buffer == NULL)
    {
        perror("Memory allocation failed");
        return 1;
    }

    pthread_t producer_thread;
    pthread_t consumer_thread;

```

```

GNU nano 7.2                                     pc_semaphore.c
sem_init(&empty, 0, buffer_size);
sem_init(&full, 0, 0);

/* Initialize mutex */
pthread_mutex_init(&mutex, NULL);

struct timespec start, end;

/* Start timer */
clock_gettime(CLOCK_MONOTONIC, &start);

/* Create threads */
pthread_create(&producer_thread, NULL, producer, NULL);
pthread_create(&consumer_thread, NULL, consumer, NULL);

/* Wait for threads */
pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);

/* Stop timer */
clock_gettime(CLOCK_MONOTONIC, &end);

double elapsed =
    (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1e9;

double throughput = consumed / elapsed;

printf("\n");

```

```

printf("\n");
printf("=====\n");
printf("          PRODUCER-CONSUMER RESULTS\n");
printf("=====\n");
printf("Buffer Size      : %d\n", buffer_size);
printf("Items Produced   : %lld\n", produced);
printf("Items Consumed    : %lld\n", consumed);
printf("Execution Time    : %.6f seconds\n", elapsed);
printf("Throughput       : %.2f items/second\n", throughput);

if (produced == consumed)
    printf("Synchronization : CORRECT\n");
else
    printf("Synchronization : INCORRECT\n");

printf("=====\n");

/* Destroy synchronization objects */
sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

/* Free memory */
free(buffer);

return 0;
}

```

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```

divya@Divyasree:~$ mkdir deadlock_demo
divya@Divyasree:~$ cd deadlock_demo
divya@Divyasree:~/deadlock_demo$ pwd
/home/divya/deadlock_demo
divya@Divyasree:~/deadlock_demo$ nano deadlock_demo.c
divya@Divyasree:~/deadlock_demo$ gcc deadlock_demo.c -o deadlock_demo -pthread
divya@Divyasree:~/deadlock_demo$ ls
deadlock_demo  deadlock_demo.c
divya@Divyasree:~/deadlock_demo$ ./deadlock_demo
=====
DEADLOCK DEMONSTRATION
=====
Thread 1: Trying to acquire Resource 1...
Thread 1: Acquired Resource 1.
Thread 2: Trying to acquire Resource 2...
Thread 2: Acquired Resource 2.
Thread 1: Trying to acquire Resource 2...
Thread 2: Trying to acquire Resource 1...
^C
divya@Divyasree:~/deadlock_demo$ nano deadlock_prevention.c
divya@Divyasree:~/deadlock_demo$ gcc deadlock_prevention.c -o deadlock_prevention -pthread
divya@Divyasree:~/deadlock_demo$ ls
deadlock_demo  deadlock_prevention
deadlock_demo.c  deadlock_prevention.c
divya@Divyasree:~/deadlock_demo$ ./deadlock_prevention
=====

```

```

divya@Divyasree:~/deadlock_demo$ ls
deadlock_demo  deadlock_prevention
deadlock_demo.c  deadlock_prevention.c
divya@Divyasree:~/deadlock_demo$ ./deadlock_prevention
=====
DEADLOCK PREVENTION
RESOURCE ORDERING
=====

Thread 1: Acquiring Resource 1...
Thread 1: Acquired Resource 1.
Thread 2: Acquiring Resource 1...
Thread 1: Acquiring Resource 2...
Thread 1: Acquired Resource 2.
Thread 1: Using both resources.
Thread 1: Released both resources.
Thread 2: Acquired Resource 1.
Thread 2: Acquiring Resource 2...
Thread 2: Acquired Resource 2.
Thread 2: Using both resources.
Thread 2: Released both resources.

=====
No deadlock occurred.
Resource ordering was successful.
=====
divya@Divyasree:~/deadlock_demo$ |

```

Code:

```

GNU nano 7.2                                deadlock_demo.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

/* Thread 1 */
void *thread1(void *arg)
{
    printf("Thread 1: Trying to acquire Resource 1...\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Acquired Resource 1.\n");
    sleep(1);

    printf("Thread 1: Trying to acquire Resource 2...\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Acquired Resource 2.\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

/* Thread 2 */
void *thread2(void *arg)

```

```
GNU nano 7.2 deadLock_demo.c
{
    printf("Thread 2: Trying to acquire Resource 2...\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Acquired Resource 2.\n");
    sleep(1);

    printf("Thread 2: Trying to acquire Resource 1...\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Acquired Resource 1.\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    printf("=====\n");
    printf("      DEADLOCK DEMONSTRATION\n");
    printf("=====\\n\\n");
```

```
    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("\\nProgram completed.\\n");

    return 0;
}
```

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

/* Thread 1 */
void *thread1(void *arg)
{
    printf("Thread 1: Acquiring Resource 1...\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Acquired Resource 1.\n");

    sleep(1);

    printf("Thread 1: Acquiring Resource 2...\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Acquired Resource 2.\n");

    printf("Thread 1: Using both resources.\n");

    sleep(1);

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

```

```

    printf("Thread 1: Released both resources.\n");

    return NULL;
}

/* Thread 2 */
void *thread2(void *arg)
{
    /*
     * Resource ordering:
     * Resource 1 must always be acquired before Resource 2.
     */

    printf("Thread 2: Acquiring Resource 1...\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Acquired Resource 1.\n");

    sleep(1);

    printf("Thread 2: Acquiring Resource 2...\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Acquired Resource 2.\n");

    printf("Thread 2: Using both resources.\n");

    sleep(1);

```

```

pthread_mutex_unlock(&resource2);
pthread_mutex_unlock(&resource1);

printf("Thread 2: Released both resources.\n");

return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    printf("=====\n");
    printf("    DEADLOCK PREVENTION\n");
    printf("    RESOURCE ORDERING\n");
    printf("=====\n\n");

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

```

```

pthread_mutex_destroy(&resource1);
pthread_mutex_destroy(&resource2);

printf("\n=====\n");
printf("No deadlock occurred.\n");
printf("Resource ordering was successful.\n");
printf("=====\n");

return 0;
}

```